



BEE HOUSE · 벌집게임

AI 활용 기술 문서

AI 도구 · 대표 프롬프트 · 활용 내역 · 검증 근거

사람은 게임의 방향과 품질 기준을 결정하고,
AI는 분석·구현·검증의 반복을 가속했습니다.

문서 기준일 2026. 07. 20

게임 버전 3.2.5+325

활용 도구 OpenAI Codex

1. AI 활용 요약

Bee House는 생성형 AI가 플레이 도중 정답이나 콘텐츠를 실시간으로 만드는 게임이 아니다. 제작 과정에서 OpenAI Codex를 저장소 분석, 기능 구현, 리팩터링, 오류 대응, 테스트, 성능 개선, 기술 문서화를 돕는 개발 파트너로 활용했다.

게임의 핵심 아이디어, 컬러 바와 육각 경로 규칙, 점수·시간 정책, 사용자 경험, 비주얼 방향, 기능 채택과 출시 여부는 개발자가 결정했다. AI는 개발자의 목표를 구현 가능한 작업으로 나누고, 관련 코드를 찾아 수정하며, 정상 흐름뿐 아니라 네트워크 실패·앱 종료·중복 제출과 같은 예외 상황과 회귀 가능성을 함께 점검했다.

런타임 게임 로직은 외부 AI API에 의존하지 않는다. 오늘의 퍼즐과 리플레이는 같은 조건을 재현해야 하므로 시드와 결정론적 규칙으로 계산한다. 따라서 AI 서비스의 응답 변화, 장애, 지연 또는 비용이 게임 규칙과 랭킹의 공정성에 영향을 주지 않는다.

2. 활용한 AI 도구

도구	활용 분야	AI에 제공한 정보	주요 산출물
OpenAI Codex	코드베이스 탐색, Flutter/Dart 구현, Supabase SQL 검토, 리팩터링, 테스트, 빌드, 문서 작성	기능 목표, 현재 증상, 변경 금지 규칙, 소스 코드, 오류 로그, 테스트 결과, 제출 요구사항	코드 수정안, SQL migration, 자동 테스트, 검증 결과, 기술 문서 초안

확인 가능한 개발 기록을 기준으로 본 문서에 기재한 생성형 AI 도구는 OpenAI Codex다. 다른 생성형 AI 도구의 사용 여부를 확인할 수 있는 근거가 없어 임의로 추가하지 않았다.

일반적인 사용 절차

- 개발자가 해결할 문제와 유지해야 할 게임 규칙을 자연어로 설명한다.
- Codex가 관련 코드, 문서, 테스트와 Git 변경 이력을 확인한다.
- 현재 동작과 문제 원인, 수정 범위와 회귀 위험을 정리한다.
- 코드·SQL·테스트 또는 문서 수정안을 작성한다.
- `flutter analyze`, `flutter test`, 릴리스 빌드를 실행한다.
- 개발자가 코드 차이와 실제 플레이 결과를 검토하고 채택 또는 재수정을 지시한다.

AI는 제품 결정을 독립적으로 내리는 자동 개발자가 아니라, 개발자의 지시 아래 탐색과 반복 작업의 범위를 넓혀 주는 협업 도구로 사용했다.

3. 사람과 AI의 역할 분담

영역	사람이 담당한 역할	AI가 지원한 역할
게임 기획	핵심 재미, 컬러 바·육각 경로 규칙, 기능 우선순위 결정	규칙을 구현 단위로 분해하고 코드·문서의 일치 여부 점검
게임 밸런스	제한 시간, 점수·시간·콤보 정책 결정과 최종 승인	계산식 구현 위치 탐색, 경계 조건과 회귀 위험 확인
UI/UX	비주얼 방향, 정보 우선순위, 조작감과 최종 사용성 판단	위젯 구조 분석, 반응형 레이아웃과 접근성 개선안 구현
클라이언트	기능 범위, 플랫폼 정책, 실제 기기 동작 확인	Flutter/Dart 구현, 파일 분리, 오류 원인 분석
서버·데이터	점수 정책, 인증·권한 범위, 운영 기준 승인	Supabase RPC·RLS·migration 수정안 및 실패 흐름 점검
품질 검증	최종 플레이 감각과 출시 판단	정적 분석, 자동 테스트, 릴리스 빌드와 결과 요약
출품 준비	제출 사실 확인, 에셋 권리 확인, 최종 문서 승인	요구사항 정리, 소개서·기술서 원고와 체크리스트 작성

4. 프롬프트 설계 방식

단순히 “게임을 만들어 줘”라고 요청하지 않고, AI가 지켜야 할 목표, 현재 문제, 유지할 규칙, 작업 범위, 금지 사항, 완료 조건을 함께 전달했다.

[목표]

무엇을 구현하거나 개선해야 하는지 설명한다.

[현재 상황]

사용자가 겪는 문제, 재현 조건, 관련 오류를 설명한다.

[유지할 규칙]

점수, 시간, 매칭, 데이터 정책 등 바꾸면 안 되는 동작을 명시한다.

[작업 범위]

확인할 클라이언트 코드, 서버 로직, UI, 테스트 범위를 정한다.

[금지 사항]

비밀키 노출, 무관한 기능 변경, 검증되지 않은 추정 등을 금지한다.

[완료 조건]

정적 분석, 자동 테스트, 빌드와 사람이 확인할 항목을 명시한다.

이 구조를 사용하면 AI가 문제를 임의로 재해석하거나 버그 수정과 함께 핵심 밸런스를 바꾸는 일을 줄일 수 있다. 또한 결과를 코드 차이와 실행 기록으로 확인하기 쉬워진다.

5. 대표 프롬프트와 활용 결과

아래 프롬프트는 실제 개발 지시의 목적을 심사자가 이해하기 쉽게 재구성한 요약본이다. 긴 대화에 포함된 코드 문맥과 중간 실행 로그는 생략했으며, 원문을 그대로 복제한 대화록은 아니다.

사례 1. 핵심 규칙을 보존한 구조 리팩터링

대표 프롬프트

[목표] 게임 화면과 컨트롤러의 책임을 분리해 유지보수성을 높여 줘.

[유지할 규칙] 컬러 바 매칭, 드래그, 점수, 시간, 콤보와 현재 화면 동작은 바꾸지 마.

[작업 범위] 게임 실행 흐름, 보드, 드래그, 라운드 진행, 제출 흐름을 점검해.

[완료 조건] 책임 단위로 파일을 분리하고 정적 분석과 전체 테스트를 통과시켜 줘.

AI 활용 결과

- 게임 실행과 결과 처리를 `GameRunCoordinator` 중심으로 분리했다.
- 드래그, 매칭, 라운드, 보드 표시 책임을 작은 파일로 나눴다.
- 중복·레거시 위젯과 불필요한 의존성을 정리했다.
- 핵심 규칙은 `docs/game_rules.md`, 리플레이 규칙은 `docs/replay_rules.md`에 고정 기준으로 기록했다.

확인 근거: `e8f5d1f, 9fd034c` 커밋, `lib/screens/game/game_run_coordinator.dart`, `docs/game_rules.md`, `docs/replay_rules.md`

사례 2. 오늘의 퍼즐 제출 실패 복구

대표 프롬프트

[목표] 오늘의 퍼즐은 하루 한 번만 공식 점수를 제출하게 하되 미완료 시도는 복구해 줘.

[현재 상황] 앱 종료나 네트워크 오류 뒤 사용자의 도전권이 영구 잠길 수 있어.

[유지할 규칙] 완료 점수의 중복 제출은 차단하고 날짜별 공식 기록 정책은 유지해.

[작업 범위] 클라이언트 진입 조건, 로컬 대기열, 서버 RPC와 RLS를 함께 확인해.

[완료 조건] 성공·실패·재시도 시나리오를 테스트로 검증해 줘.

AI 활용 결과

- 도전권 사용 여부와 서버 점수 제출 완료 여부를 분리해 판단했다.
- 제출되지 않은 공식 시도는 다시 열고, 이미 저장된 시도는 차단하도록 수정했다.
- 네트워크 실패 시 점수를 로컬에 보관하고 다음 실행에서 재제출하는 서비스를 추가했다.
- 서버 변경을 날짜가 포함된 SQL migration으로 남겼다.
- 성공 제출, 실패 후 보존, 재시도 흐름을 자동 테스트로 검증했다.

확인 근거: b7c4801, 9e2c5b7 커밋, lib/services/daily_submission_service.dart, test/game_run_coordinator_test.dart, supabase/migrations/202604140002_allow_retry_for_incomplete_daily_attempts.sql

사례 3. 결정론적 오늘의 퍼즐과 리플레이 검증

대표 프롬프트

[목표] 같은 날짜의 오늘의 퍼즐과 같은 리플레이 입력이 항상 같은 결과를 내게 해 줘.
 [유지할 규칙] 보드와 컬러 바 생성에는 하나의 시드 흐름을 사용하고 점수식을 바꾸지 마.
 [작업 범위] 난수 생성, 게임 초기화, 성공 경로 기록, 서버 검증 payload를 확인해.
 [금지 사항] 리플레이 중 사용자 입력이나 타이머가 결과에 개입하면 안 돼.
 [완료 조건] 골든 시퀀스와 리플레이 직렬화 테스트를 추가해 줘.

AI 활용 결과

- 클라이언트와 서버가 공유할 수 있는 결정론적 난수 순서를 테스트했다.
- 성공 경로, 콤보와 권위 있는 시간 정보를 리플레이 payload에 기록했다.
- 서버가 비정상 점수 기록을 다시 계산해 검증할 수 있는 migration을 구성했다.
- 일반 모드와 Honey Rush 리플레이 형식을 자동 테스트로 확인했다.

확인 근거: test/deterministic_random_test.dart, test/replay_share_service_test.dart, supabase/migrations/202607140003_authoritative_hexor_replay_verifier.sql, supabase/migrations/202607150002_verify_honey_rush_replays.sql

사례 4. 반응형 UI·다국어·접근성 개선

대표 프롬프트

[목표] 휴대폰과 iPad, 모든 지원 언어에서 핵심 게임 정보가 화면 안에 보여야 해.
 [유지할 규칙] 현재 디자인 언어와 플레이 우선순위는 유지해.
 [작업 범위] 컬러 바, 타이머, 점수, 보드, 결과 화면과 번역 키를 점검해.
 [완료 조건] 작은 화면과 태블릿 비율의 위젯 테스트, 번역 누락 테스트를 통과시켜 줘.

AI 활용 결과

- 사용 가능한 화면 높이에 맞춰 HUD와 보드의 크기를 계산하도록 레이아웃을 정리했다.
- 게임 종료 카드가 작은 화면 밖으로 넘치지 않도록 제약을 강화했다.
- 모든 지원 언어의 번역 키 완전성과 코드 내 번역 키 사용을 테스트했다.
- 보드 타일에 텍스트 방향이 포함된 semantic label을 제공해 접근성 회귀를 방지했다.

확인 근거: 8b378db, 664300b 커밋, test/game_hud_test.dart, test/game_over_overlay_test.dart, test/app_translations_test.dart, test/hex_board_semantics_test.dart

사례 5. Honey Rush와 게임 규칙 회귀 방지

대표 프롬프트

[목표] 10칸 매칭 보상인 Honey Rush가 점수를 중복 계산하지 않도록 구현을 점검해 줘.
 [유지할 규칙] 일반 점수·콤보 계산식과 오늘의 퍼즐 공식 경쟁 조건은 유지해.
 [작업 범위] 라운드 전환, 타일 수집, HUD 표시, 리플레이 payload를 확인해.
 [완료 조건] 10칸 매칭, 공식 일일 모드 비활성화, 리플레이 형식 테스트를 추가해 줘.

AI 활용 결과

- 10칸 성공 시 Honey Rush 진입과 타일별 점수 계산을 분리했다.

- 공식 오늘의 퍼즐에서는 Honey Rush가 커지지 않도록 모드 조건을 고정했다.
- Honey Rush 서버 리플레이 형식을 별도 테스트로 검증했다.

확인 근거: 664300b 커밋, `test/hex_game_controller_test.dart`, `test/replay_share_service_test.dart`

사례 6. 성능과 데이터 조회 최적화

대표 프롬프트

[목표] 게임 중 불필요한 화면 재계산과 일일 랭킹 달력의 반복 서버 요청을 줄여 줘.
 [유지할 규칙] 화면에 보이는 점수와 랭킹 결과는 이전과 같아야 해.
 [작업 범위] 게임 HUD 갱신, 홈 달력 데이터 로딩, Supabase 조회 경로를 분석해.
 [완료 조건] 중복 작업을 제거하고 기존 테스트와 정적 분석을 통과시켜 줘.

AI 활용 결과

- 게임 화면에서 반복되던 계산과 불필요한 갱신 범위를 줄였다.
- 일일 랭킹을 여러 번 조회하는 대신 배치 RPC로 가져오는 migration과 클라이언트 호출을 추가했다.
- 성능 변경이 UI와 데이터 결과를 바꾸지 않는지 기존 테스트로 확인했다.

확인 근거: 5711ca4 커밋, `lib/services/database_service.dart`,
`supabase/migrations/202607150004_batch_daily_ranks.sql`

사례 7. 공모전 제출 자료 작성

대표 프롬프트

[목표] 저장소와 제출 요구사항을 비교해 게임 소개서와 AI 활용 기술서를 작성해 줘.
 [작업 범위] 실제 게임 규칙, 실행 방법, AI 도구, 대표 프롬프트, 활용 내역,
 사람과 AI의 역할, 검증 방법, 외부 에셋 확인 항목을 구분해.
 [금지 사항] 코드나 기록으로 확인할 수 없는 내용을 추정해서 채우지 마.
 [완료 조건] Markdown 원고와 PDF를 만들고 렌더링 결과를 확인해 줘.

AI 활용 결과

- 소스 코드, 규칙 문서, 테스트와 변경 이력을 대조해 제출용 원고를 작성했다.
- 실제 개발 지시를 목적 중심의 대표 프롬프트로 정리했다.
- 이미지·음원의 출처처럼 저장소에서 확정할 수 없는 항목은 별도 확인표로 분리했다.
- 정적 분석, 전체 테스트, 웹 릴리스 빌드를 다시 실행해 최신 결과를 반영했다.

확인 근거: `docs/nan2026/`, `output/pdf/`, 2026년 7월 20일 검증 기록

6. AI 활용 내역 일람

개발 단계	AI 활용 내용	적용 결과	검증·증빙
기획 정리	자연어 규칙을 구현 조건과 변경 금지 기준으로 구조화	게임·리플레이 규칙 문서화	docs/game_rules.md, docs/replay_rules.md
코드 분석	화면, 컨트롤러, 서비스, SQL 사이의 의존 관계 탐색	수정 영향 범위와 회귀 지점 파악	Git diff, 관련 파일 목록
기능 구현	오늘의 퍼즐 복구, Honey Rush, 리플레이, 랭킹 흐름 구현 지원	사용자 기능과 실패 복구 흐름 개선	기능별 소스와 migration
리팩터링	큰 파일과 혼합된 책임을 작은 단위로 분리	유지보수성과 테스트 가능성 향상	e8f5d1f 등 변경 이력
서버·보안	RPC, RLS, 점수 제출과 리플레이 검증 흐름 점검	중복 제출 방지, 서버 권위 검증 강화	supabase/migrations/
UI/UX	반응형 제약, 번역, 접근성, 디자인 일관성 점검	휴대폰·태블릿·다국어 대응 강화	위젯·번역·semantic 테스트
성능	반복 렌더링과 여러 번의 데이터 요청 탐색	게임·달력 작업량과 조회 횟수 감소	5711ca4, 배치 RPC
품질 검증	분석, 테스트, 빌드 실행과 실패 원인 추적	최신 기준 분석 통과, 테스트 33건 통과, 웹 빌드 성공	명령 실행 결과, test/
문서화	코드·문서·커밋에서 사실을 수집해 제출 원고 구성	소개서, AI 기술서, 체크리스트 작성	docs/nan2026/, output/pdf/

7. 기술 영역별 적용 내용

영역	기술	AI 활용 내용
앱	Flutter / Dart	화면·게임 로직 분석, 기능 구현, 리팩터링
상태 관리	GetX	컨트롤러 의존 관계와 화면 갱신 흐름 점검
백엔드	Supabase Auth / PostgreSQL	프로필·점수·랭킹 데이터 흐름 분석
서버 로직	RPC / Row Level Security	제출 원자성, 권한, 실패 복구와 점수 검증 검토
재현성	시드 기반 난수 생성	오늘의 퍼즐과 리플레이의 동일 초기 상태 보장
로컬 데이터	Shared Preferences	설정·진행·제출 대기 데이터 관리
검증	Flutter analyze / test / build	변경 후 정적 오류, 회귀와 빌드 가능 여부 확인
문서	Markdown / Git	규칙, 프롬프트, 활용 내역과 변경 근거 관리

8. 최신 검증 결과

2026년 7월 20일, 로컬 **main** 브랜치와 현재 작업 트리 기준으로 다음 명령을 실행했다.

```
flutter analyze
flutter test
flutter build web --release
```

검증 항목	결과
Flutter 정적 분석	오류·경고 없음
자동 테스트	33건 전체 통과
웹 릴리스 빌드	성공, build/web 생성

자동 테스트는 결정론적 난수, 점수·시간·콤보, Honey Rush, 오늘의 퍼즐 제출과 재시도, 리플레이 payload, 반응형 HUD·결과 화면, 다국어 번역, 접근성 semantic label, 시즌 티어 등을 확인한다.

자동 검증은 실제 플레이 감각을 대신할 수 없다. 화면의 인상, 손가락 드래그 감도, 사운드·햅틱과 최종 밸런스는 개발자가 실제 기기에서 확인한다.

9. 검증과 안전장치

핵심 규칙 보호

보드 구성, 최소 매칭 길이, 점수식, 시간식, 콤보 판정과 종료 조건을 **docs/game_rules.md**에 명시했다. AI 작업에는 이 문서를 변경 금지 기준으로 제공하고, 규칙 변경이 필요하면 개발자가 먼저 승인한다.

코드 차이 검토

AI가 작성한 결과를 답변만 보고 채택하지 않는다. 개발자가 변경 파일과 diff를 확인해 요청 범위를 벗어난 수정, 기존 기능 삭제와 민감 정보 포함 여부를 검토한다.

자동 검증

정적 분석, 단위·위젯 테스트, 릴리스 빌드를 통해 문법·타입 오류, 주요 기능 회귀와 빌드 실패를 확인한다. 실패하면 실행 로그를 다시 AI에 제공해 원인을 좁히고 수정한다.

데이터와 보안

- Supabase 접근 권한은 RLS 정책과 RPC로 제한한다.
- 실제 점수는 서버 측 검증 로직을 통해 신뢰성을 확인한다.
- 민감한 환경 값은 빌드 환경 값으로 주입하며 프롭트와 공개 저장소에 포함하지 않는다.
- 데이터베이스 변경은 날짜가 포함된 SQL migration으로 남겨 재현과 검토가 가능하게 한다.

런타임 AI 비의존

외부 생성형 AI API를 게임 실행의 필수 조건으로 두지 않는다. 모델 응답의 비결정성, API 장애·지연·비용이 플레이 가능 여부와 공정한 랭킹에 영향을 주지 않는다.

10. AI 활용 효과

- 저장소 전체에서 관련 코드를 찾고 영향 범위를 파악하는 시간을 줄였다.
- 클라이언트, 서버, SQL, 테스트와 문서를 한 흐름으로 검토할 수 있었다.
- 정상 동작뿐 아니라 네트워크 실패, 앱 종료, 중복 제출과 같은 예외 상황을 더 넓게 확인했다.
- 큰 파일을 단계적으로 분리하면서 매 단계 분석과 테스트를 반복할 수 있었다.

- 반응형 화면, 다국어와 접근성처럼 여러 조건의 조합을 자동 테스트로 고정했다.
- 구현 내용과 제출 문서 사이의 불일치를 코드 기준으로 줄였다.

AI 활용의 핵심 성과는 개발자의 판단을 대체한 것이 아니라, 더 넓은 범위를 더 자주 검증할 수 있게 만든 것이다.

11. 한계와 개선 방향

- AI는 사용자의 재미와 손맛을 직접 판단할 수 없으므로 게임 밸런스와 조작감은 사람이 플레이하며 결정한다.
- AI가 코드 문맥을 잘못 해석할 수 있으므로 핵심 규칙 문서, 코드 차이 검토와 자동 테스트가 필요하다.
- 자동 테스트가 통과해도 모든 실제 기기, 네트워크 상태와 운영 데이터 상황을 완전히 재현할 수는 없다.
- 앞으로 서버 검증 migration의 통합 테스트, 플랫폼별 UI 테스트와 실패 복구 시나리오를 추가할 계획이다.
- 외부 이미지와 음원의 제작·구매·라이선스 정보는 코드만으로 확인할 수 없으므로 원본과 증빙을 사람이 직접 확인해야 한다.

12. 외부 에셋과 오픈소스 관리

프로젝트의 주요 직접 의존성은 GetX, audioplayers, shared_preferences, google_fonts, supabase_flutter, google_sign_in, sign_in_with_apple, crypto, url_launcher, google_mobile_ads, share_plus다. 정확한 버전은 [pubspec.yaml](#)과 [pubspec.lock](#)에 기록되어 있으며 각 프로젝트의 라이선스를 따른다.

이미지와 BGM의 출처 및 권리 근거는 [docs/nan2026/asset_provenance_checklist.md](#)에서 별도로 관리한다. 최종 제출 전 각 파일을 다음 중 하나로 확정하고 증빙을 기재해야 한다.

- 직접 제작: 제작자, 제작일, 원본 파일
- AI 생성: 도구·모델, 생성일, 대표 프롬프트, 상업 이용 조건
- 라이선스 구매: 서비스·에셋 URL, 라이선스명, 구매 증빙
- 무료 또는 제3자 제공: 제작자, 원문 URL, 라이선스와 표시 조건

저장소만으로는 현재 이미지·음원의 권리 상태를 확인할 수 없으므로 본 문서에서 임의로 기재하지 않았다. 이 항목은 최종 제출 전에 개발자가 반드시 확인해야 한다.

13. 재현 가능한 근거

- 전체 소스 코드: [GitHub 저장소](#)
- 핵심 게임 규칙: [docs/game_rules.md](#)
- 리플레이 규칙: [docs/replay_rules.md](#)
- 자동 테스트: [test/](#)
- 데이터베이스 변경 이력: [supabase/migrations/](#)
- 공모전 제출 자료: [docs/nan2026/](#)
- AI 협업이 명시된 변경 이력: [b7c4801](#), [9e2c5b7](#) 및 [codex/daily-challenge-retry](#) 브랜치
- 최근 구조·품질 개선 이력: [e8f5d1f](#), [5711ca4](#), [664300b](#)

14. 결론

Bee House에서 AI는 게임 규칙을 대신 결정하거나 플레이 결과를 임의로 생성하지 않았다. 개발자가 핵심 경험과 품질 기준을 정의하고, OpenAI Codex를 이용해 저장소 분석, 기능 구현, 리팩터링, 예외 상황 점검, 테스트와 문서화를 반복했다.

그 결과 결정론적 게임 규칙과 공정한 랭킹은 유지하면서 오늘의 퍼즐 복구, 서버 검증, 반응형 UI, 다국어·접근성, 성능 개선과 제출 문서화를 더 체계적으로 수행할 수 있었다. 이 과정은 소스 코드, SQL migration, 테스트와 Git 변경 이력으로 확인할 수 있다.